

Desenvolvimento Web com .NET

Aula 3

Prof. Me. Lucas Teodoro dos Santos

UNIP

26 de Agosto de 2026

- 1 ORM
- 2 Entity Framework Core
- 3 Migrations

Dois Paradigmas Diferentes

Em nossa aplicação ASP.NET Core, trabalhamos seguindo o paradigma **orientado a objetos**.

Criamos classes, instanciamos objetos e acessamos suas propriedades:

```
1 Medico medico = new Medico
2 {
3     Id = 1,
4     Nome = "Ana Souza",
5     Crm = "12345",
6     Especialidade = "Cardiologia"
7 };
8
9 Console.WriteLine(medico.Nome);
```

Porém, o PostgreSQL trabalha seguindo o modelo **relacional**, com tabelas, colunas, registros e chaves.

Desafio

Como transformar os dados das tabelas em objetos que possam ser utilizados pela aplicação?

Consultando Diretamente com SQL

Para consultar os médicos cadastrados diretamente no banco, podemos escrever um comando SQL:

```
SELECT "Id", "Nome", "Crm", "Especialidade"  
FROM "Medicos"  
WHERE "Especialidade" = 'Cardiologia';
```

O PostgreSQL retorna registros em formato tabular:

Id	Nome	Crm	Especialidade
1	Ana Souza	12345	Cardiologia

Mas a aplicação não trabalha naturalmente com uma linha de tabela: ela precisa de um objeto Medico.

Transformando o Registro em Objeto

Sem utilizar um recurso apropriado, o desenvolvedor precisa ler cada registro retornado pelo banco e preencher manualmente um objeto:

```
1      var medico = new Medico
2      {
3          Id = reader.GetInt32(0),
4          Nome = reader.GetString(1),
5          Crm = reader.GetString(2),
6          Especialidade = reader.GetString(3)
7      };
```

Esse processo é chamado de **mapeamento objeto-relacional**:

Conversão necessária

Registro da tabela → objeto Medico

Em consultas maiores, também seria necessário abrir a conexão, executar o comando, percorrer todos os registros e tratar possíveis erros.

Qual a solução para esse problema?

A aplicação precisa trabalhar com objetos, mas os dados estão armazenados em tabelas no banco de dados.

Portanto, em cada operação é necessário realizar uma conversão entre esses dois modelos:

Mapeamento necessário

Objetos C# $\longleftrightarrow$ tabelas e registros do banco de dados

Se essa conversão for feita manualmente, o desenvolvedor precisará escrever comandos SQL, executar consultas e transformar os resultados em objetos.

Esse processo pode gerar repetição de código e tornar a manutenção da aplicação mais trabalhosa.

Solução proposta

Utilizar uma ferramenta capaz de realizar esse mapeamento entre a aplicação e o banco de dados.

O que é um ORM?

Um ORM é uma ferramenta que realiza o mapeamento entre os objetos da aplicação e as estruturas de um banco de dados relacional.

A sigla ORM significa:

Object-Relational Mapper

Mapeador Objeto-Relacional

Com um ORM, o desenvolvedor pode trabalhar com classes e objetos, enquanto a ferramenta realiza grande parte da comunicação com o banco de dados.

O ORM transforma operações realizadas nos objetos em comandos que podem ser compreendidos pelo banco.

Funcionamento de um ORM

Considere que a aplicação precisa consultar os médicos cadastrados.

Sem um ORM, seria necessário:

- ❶ Criar o comando SQL;
- ❷ Abrir uma conexão com o banco;
- ❸ Executar o comando;
- ❹ Percorrer o resultado;
- ❺ Transformar cada registro em um objeto.

Com um ORM, parte desse processo é realizada automaticamente, permitindo que o resultado seja obtido diretamente como uma coleção de objetos.

ORMs em Diferentes Linguagens

O mapeamento entre objetos e bancos relacionais é uma necessidade comum em diversas linguagens de programação. Por isso, existem várias ferramentas ORM.

Linguagem	Exemplos de ORM
C# / .NET	Entity Framework Core, Dapper
Java	Hibernate, JPA
Python	Django ORM, SQLAlchemy
JavaScript / TypeScript	Prisma, TypeORM, Sequelize
PHP	Eloquent, Doctrine
Ruby	Active Record

Apesar de possuírem APIs e recursos diferentes, essas ferramentas têm o mesmo objetivo: facilitar a comunicação entre objetos da aplicação e dados armazenados em bancos relacionais.

Entity Framework Core

O Entity Framework Core, também chamado de EF Core, é o principal ORM utilizado em aplicações .NET.

Ele funciona como uma camada entre a aplicação e o banco de dados:

Comunicação

Aplicação C# $\longleftrightarrow$ EF Core $\longleftrightarrow$ Banco de dados

O EF Core analisa as classes da aplicação, realiza o mapeamento dos dados e gera os comandos necessários para consultar ou modificar o banco.

Ele pode ser utilizado com PostgreSQL, SQL Server, SQLite e outros bancos, por meio de provedores específicos.

O que o EF Core permite fazer?

Com o Entity Framework Core, podemos:

- Representar tabelas do banco por meio de classes C#;
- Consultar dados e receber o resultado como objetos;
- Adicionar, alterar e excluir registros;
- Representar relacionamentos entre diferentes entidades;
- Acompanhar as alterações realizadas nos objetos;
- Criar e atualizar a estrutura do banco de dados;
- Trabalhar com diferentes sistemas de banco de dados.

Vantagens do Entity Framework Core

A utilização do EF Core oferece algumas vantagens:

- **Produtividade:** Reduz a quantidade de código necessário para acessar o banco;
- **Organização:** Centraliza o mapeamento e o acesso aos dados;
- **Manutenção:** Facilita a evolução da estrutura do banco;
- **Tipagem:** Permite trabalhar com classes e propriedades C#;
- **Portabilidade:** Pode utilizar diferentes bancos por meio de provedores;
- **Integração:** Funciona de forma integrada com aplicações ASP.NET Core.

O EF Core Substitui o SQL?

O EF Core reduz a necessidade de escrever manualmente comandos SQL, mas não elimina a importância de conhecer banco de dados.

O desenvolvedor ainda precisa compreender:

- Tabelas, colunas e tipos de dados;
- Chaves primárias e estrangeiras;
- Relacionamentos;
- Consultas e filtros;
- Índices e desempenho;
- Transações e integridade dos dados.

Importante

O EF Core facilita o acesso aos dados, mas o banco continua executando os comandos e armazenando as informações.

Exemplo de Mapeamento

```
1 public class Medico
2 {
3     public int Id { get; set; }
4
5     public string Nome { get; set; } = string.Empty;
6
7     public string Crm { get; set; } = string.Empty;
8
9     public string Especialidade { get; set; } = string.Empty;
10 }
```

Por convenção, o EF Core pode mapear essa classe da seguinte maneira:

- A classe Medico representa uma entidade;
- A entidade será associada a uma tabela;
- As propriedades serão associadas às colunas;
- Cada objeto representará um registro;
- A propriedade Id será reconhecida como chave primária.

Exemplo de Mapeamento

```
1 public class Medico
2 {
3     public int Id { get; set; }
4     public string Nome { get; set; } = string.Empty;
5     public string Crm { get; set; } = string.Empty;
6     public string Especialidade { get; set; } = string.Empty;
7 }
```

A partir dessa classe, o EF Core pode criar a tabela Medicos, na qual cada propriedade é representada por uma coluna.

Medicos:

Id	Nome	Crm	Especialidade
1	Ana Souza	12345	Cardiologia
2	Carlos Lima	67890	Ortopedia

Code First é uma abordagem na qual as classes da aplicação são criadas antes da estrutura do banco de dados.

O desenvolvedor define as entidades e seus relacionamentos utilizando C#. Depois, o EF Core utiliza essas definições para criar ou atualizar o banco.

Fluxo do Code First

Classes C# $\longrightarrow$ Migration $\longrightarrow$ Banco de dados

Assim, as classes da aplicação tornam-se a principal referência para a estrutura dos dados.

Database First é uma abordagem na qual o banco de dados é criado antes das classes da aplicação.

O banco já possui suas tabelas, colunas, chaves e relacionamentos. Depois, o EF Core analisa essa estrutura e gera as classes e o DbContext.

Fluxo do Database First

Banco de dados → EF Core → Classes C#

Essa abordagem é utilizada principalmente quando a aplicação precisa trabalhar com um banco de dados que já existe.

Code First e Database First

As duas abordagens diferem principalmente pelo ponto de partida do projeto.

Code First	Database First
As classes são criadas primeiro.	O banco de dados é criado primeiro.
O banco é gerado a partir das classes.	As classes são geradas a partir do banco.
Utiliza migrations para atualizar o banco.	Utiliza engenharia reversa para ler o banco existente.
Indicado para projetos novos.	Indicado para bancos de dados já existentes.

A abordagem Code First permite:

- Modelar o sistema utilizando classes C#;
- Manter a estrutura do banco próxima ao código da aplicação;
- Registrar as alterações do banco por meio de migrations;
- Reproduzir a estrutura do banco em outros ambientes;
- Trabalhar com controle de versão das alterações;
- Evoluir o banco conforme o projeto é desenvolvido.

Etapas do Code First

Para utilizar a abordagem Code First, normalmente seguimos estas etapas:

- ❶ Criar as classes que representam os dados;
- ❷ Criar a classe de contexto do banco;
- ❸ Configurar a conexão com o PostgreSQL;
- ❹ Registrar o contexto no `Program.cs`;
- ❺ Criar uma migration;
- ❻ Aplicar a migration no banco de dados.

Pacotes Necessários

Para utilizar o Entity Framework Core com PostgreSQL, instale os seguintes pacotes no projeto:

- `Npgsql.EntityFrameworkCore.PostgreSQL`
Permite que o Entity Framework Core se comunique com o PostgreSQL.
- `Microsoft.EntityFrameworkCore.Design`
Disponibiliza os recursos utilizados durante a criação e execução das migrations.
- `Microsoft.EntityFrameworkCore.Tools`
Disponibiliza comandos como Add-Migration e Update-Database no Visual Studio.

Atenção

Os pacotes do Entity Framework Core e do Npgsql devem utilizar versões compatíveis com a versão do .NET utilizada no projeto.

PostgreSQL e Npgsql

O PostgreSQL será utilizado como sistema gerenciador de banco de dados do projeto.

Para que o Entity Framework Core consiga se comunicar com o PostgreSQL, utilizaremos o provedor **Npgsql**.

- PostgreSQL: armazena e gerencia os dados;
- EF Core: realiza o mapeamento objeto-relacional;
- Npgsql: permite a comunicação entre o EF Core e o PostgreSQL.

Pacote principal

```
Npgsql.EntityFrameworkCore.PostgreSQL
```

Configuração no Visual Studio - Gerenciar Pacotes NuGet

No Visual Studio, abra:

Projeto → Gerenciar Pacotes NuGet

Instale os seguintes pacotes:

- `Npgsql.EntityFrameworkCore.PostgreSQL;`
- `Microsoft.EntityFrameworkCore.Tools;`
- `Microsoft.EntityFrameworkCore.Design.`

Versões

Utilize versões compatíveis entre o .NET, o Entity Framework Core e o provedor Npgsql.

Configuração no Visual Studio - Console

No Visual Studio Community, também é possível instalar os pacotes utilizando o **Console do Gerenciador de Pacotes**.

Acesse:

Ferramentas → **Gerenciador de Pacotes NuGet**
→ **Console do Gerenciador de Pacotes**

Em seguida, execute os comandos:

```
Install-Package Npgsql.EntityFrameworkCore.PostgreSQL  
Install-Package Microsoft.EntityFrameworkCore.Tools  
Install-Package Microsoft.EntityFrameworkCore.Design
```

Para instalar uma versão específica, adicione o parâmetro `-Version`:

```
Install-Package Microsoft.EntityFrameworkCore.Tools -Version 9.0.4
```


Configuração no VS Code

No VS Code, abra o terminal na pasta do projeto e execute:

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL  
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Opcional: Definir a versão

Para instalar uma versão específica, utilize `--version`:

```
dotnet add package Microsoft.EntityFrameworkCore.Design --version 9.0.4
```

Instale também a ferramenta de linha de comando do EF Core:

```
dotnet tool install --global dotnet-ef
```

Para verificar a instalação:

```
dotnet ef
```

The logo of UNIP (Universidade Nove de Julho) is displayed in yellow and red.

O DbContext é a classe central do Entity Framework Core. Ele representa a comunicação entre a aplicação e o banco de dados.

Por meio dele, a aplicação consegue consultar, inserir, alterar e excluir dados sem precisar escrever manualmente todo o código de acesso ao banco.

Ele é responsável por:

- Configurar a comunicação com o banco de dados;
- Representar as tabelas da aplicação por meio de propriedades DbSet;
- Consultar e acompanhar alterações nos objetos;
- Salvar as alterações no banco com `SaveChanges()`.

Organização

A classe de contexto normalmente é criada na pasta Data. Ela concentra a configuração e o acesso às entidades persistidas no banco.

Criando o AppDbContext

Na pasta Data, crie a classe AppDbContext:

```
1  using Microsoft.EntityFrameworkCore;
2  using ProjetoAgendamento.Models;
3
4  namespace ProjetoAgendamento.Data;
5
6  public class AppDbContext : DbContext
7  {
8      public AppDbContext(DbContextOptions<AppDbContext> options): base(options)
9      {
10      }
11  }
```

No EF Core, utilizamos `DbSet<T>` para representar um conjunto de objetos que será armazenado no banco de dados.

A letra T representa o tipo de objeto com o qual desejamos trabalhar:

```
1 DbSet<TipoDaClasse>
```

Por exemplo, se a classe for `Medico`, utilizamos:

```
1 DbSet<Medico>
```

Esse conjunto permite consultar, adicionar, alterar e remover objetos desse tipo. Em uma situação simples, ele será associado a uma tabela do banco de dados.

Classe ApplicationDbContext Completa

```
1 using Microsoft.EntityFrameworkCore;
2 using PrimeiroProjeto.Models;
3
4 namespace PrimeiroProjeto.Data
5 {
6     public class ApplicationDbContext : DbContext
7     {
8         public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options): base(options)
9         {
10         }
11
12         public DbSet<Medico> Medicos { get; set; }
13     }
14 }
```

Nessa classe, cada propriedade DbSet representa um conjunto de dados acessado pela aplicação. À medida que novas entidades forem criadas, podemos adicionar novos DbSets ao contexto:

```
1 public DbSet<Paciente> Pacientes { get; set; }
```

Configurando a String de Conexão

No arquivo `appsettings.json`, adicione a string de conexão:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Host=localhost;Port=5432;Database=Agendamento;Username=postgres;Password=senha"
  }
}
```

- Host: endereço do servidor PostgreSQL;
- Port: porta utilizada pelo PostgreSQL;
- Database: nome do banco de dados;
- Username e Password: dados de acesso.

Registrando o DbContext

No arquivo `Program.cs`, importe os namespaces:

```
1 using Microsoft.EntityFrameworkCore;  
2 using ProjetoAgendamento.Data;
```

Depois, registre o `AppDbContext`:

```
1 builder.Services.AddDbContext<AppDbContext>(options => options.UseNpgsql(  
2     builder.Configuration.GetConnectionString("DefaultConnection")));
```

O método `UseNpgsql()` configura o EF Core para utilizar o PostgreSQL.

Uma *migration* registra as alterações necessárias para manter a estrutura do banco de dados sincronizada com os Models da aplicação.

Por meio das migrations, o EF Core pode:

- Criar tabelas e colunas;
- Alterar a estrutura existente;
- Adicionar ou remover relacionamentos;
- Manter um histórico das alterações do banco.

Importante

Criar uma migration não altera imediatamente o banco de dados.

Como funciona uma Migration?

Quando criamos uma migration, o EF Core compara o estado atual dos Models com o último estado registrado no projeto.

- ❶ O desenvolvedor cria ou altera um Model;
- ❷ Cria uma migration para registrar a alteração;
- ❸ O EF Core gera os arquivos da migration;
- ❹ O desenvolvedor aplica a migration ao banco;
- ❺ A estrutura do banco é atualizada.

Exemplo

Adicionar uma propriedade ao Model pode resultar na criação de uma nova coluna na tabela.

Criando Migration no Visual Studio

No Visual Studio, abra:

Ferramentas → **Gerenciador de Pacotes NuGet**
→ **Console do Gerenciador de Pacotes**

Após criar ou alterar os Models, execute:

```
Add-Migration CriacaoInicial
```

Resultado

O EF Core gera arquivos que descrevem as alterações necessárias na estrutura do banco de dados.

Criando Migration no VS Code

No VS Code, abra o terminal dentro da pasta do projeto e execute:

```
dotnet ef migrations add CriacaoInicial
```

Importante

Nesse momento, o banco de dados ainda não é criado nem alterado. Apenas os arquivos da migration são adicionados ao projeto.

Arquivos gerados pela Migration

Após a criação da migration, será adicionada ao projeto uma pasta chamada `Migrations`.

Essa pasta normalmente contém:

- O arquivo principal da migration, com as alterações que serão aplicadas ao banco;
- Um arquivo com informações sobre o modelo gerado;
- O arquivo `AppDbContextModelSnapshot.cs`, que registra o estado mais recente da estrutura representada pelos Models.

Na próxima migration, o EF Core compara os Models atuais com o *Snapshot* para identificar quais mudanças devem ser geradas.

O arquivo principal possui duas operações:

- `Up()`: aplica as alterações ao banco;
- `Down()`: desfaz as alterações realizadas.

Aplicando Migration no Visual Studio

Após conferir os arquivos gerados, aplique todas as migrations pendentes no banco de dados:

```
Update-Database
```

Para atualizar o banco até uma migration específica:

```
Update-Database CriacaoInicial
```

Para excluir completamente o banco de dados:

```
Drop-Database
```

Atenção

O comando Drop-Database exclui todas as tabelas e dados do banco.

Aplicando Migration no VS Code

Após conferir os arquivos gerados, aplique todas as migrations pendentes no banco de dados:

```
dotnet ef database update
```

Para atualizar o banco até uma migration específica:

```
dotnet ef database update CriacaoInicial
```

Para excluir completamente o banco de dados:

```
dotnet ef database drop
```

Atenção

O comando `dotnet ef database drop` pede confirmação. Para executá-lo sem confirmação, adicione a opção `-force`.

Resultado da primeira Migration

Após executar o comando de atualização, o EF Core utiliza a migration para criar ou atualizar a estrutura do PostgreSQL.

No exemplo desenvolvido, o resultado esperado é:

- Criação do banco Agendamento, caso ele ainda não exista e o usuário possua permissão;
- Criação da tabela Medicos;
- Criação das colunas definidas no Model;
- Criação da tabela de histórico das migrations.

Tabela de controle

O EF Core utiliza a tabela `__EFMigrationsHistory` para registrar as migrations que já foram aplicadas.

Nesta aula, vimos como o Entity Framework Core facilita a comunicação entre uma aplicação orientada a objetos e um banco de dados relacional.

- A aplicação trabalha com classes e objetos, enquanto o PostgreSQL trabalha com tabelas, colunas e registros;
- Um ORM realiza o mapeamento entre esses dois modelos;
- O Entity Framework Core é o ORM utilizado no ecossistema .NET;
- No modelo *Code First*, criamos as classes primeiro e o EF Core pode gerar a estrutura do banco;
- O DbContext centraliza a comunicação com o banco, e os DbSet representam os conjuntos de dados das entidades;
- As migrations registram e aplicam as mudanças realizadas na estrutura do banco de dados.

Obrigado!

Dúvidas?